Neuron에서 Hidden Layer까지

Perceptron → Activation → Classification → Universal Approximation

이번 주의 핵심 질문

하나의 affine function

$$a = \boldsymbol{w}^\top \boldsymbol{x} + b$$

만으로는 표현할 수 있는 함수의 형태가 매우 제한적이다. 그렇다면 neural network는 어떻게 비선형적인 함수와 복잡한 분류 문제를 표현할 수 있을까?
하나의 neuron을 수식과 computational graph로 나타내고, activation function이 왜 필요한지 이해한다. 그 다음 하나의 sigmoid neuron을 이용한 binary classification을 직접 학습하고, 마지막으로 여러 neuron을 hidden layer에 배치하면 왜 훨씬 복잡한 함수를 표현할 수 있는지 살펴본다.

학습목표

이 주차가 끝나면 다음을 설명하고 직접 계산할 수 있어야 한다.

1. activation function의 역할과 비선형성이 필요한 이유를 설명한다.
2. sigmoid, tanh, ReLU의 기본 형태와 local derivative를 사용할 수 있다.
3. 하나의 sigmoid neuron을 computational graph로 나타내고 backward derivative를 계산한다.
4. 1차원 binary classification에서 decision boundary의 의미를 설명한다.
5. single neuron이 표현할 수 없는 패턴을 설명하고 hidden layer의 필요성을 이해한다.
6. one-hidden-layer network가 비선형 함수를 근사하는 구조를 설명한다.
7. Universal Approximation Theorem의 의미와 한계를 설명한다.

Artificial Intelligence, Machine Learning, Deep Learning

인공지능(Artificial Intelligence, AI)은 인간이 수행하는 인식, 판단, 예측, 문제 해결과 같은 지적인 작업을 컴퓨터가 수행하도록 만드는 연구 분야를 넓게 가리킨다. 인공지능을 구현하는 방법에는 여러 가지가 있으며, 그중 하나가 machine learning이다.

Machine learning은 사람이 모든 규칙을 직접 프로그램하는 대신, 데이터로부터 모델의 규칙이나 parameter를 학습하도록 하는 방법이다. 즉, 데이터

$$(x_i, t_i), \qquad i = 1, \ldots, N$$

가 주어졌을 때, 어떤 모델

$$f(x; \theta)$$

의 parameter θ를 데이터로부터 결정하는 것이 machine learning의 가장 기본적인 형태라고 볼 수 있다.

Artificial neural network(ANN)는 machine learning에 사용되는 모델의 한 종류이다. 여러 개의 artificial neuron을 서로 연결하여 하나의 복잡한 함수를 구성한다.

그중에서도 여러 개의 layer를 갖는 neural network를 이용하여 데이터로부터 복잡한 representation을 학습하는 방법을 deep learning이라고 한다.

따라서 개념적인 포함관계는

$$\boxed{\text{Artificial Intelligence} \supset \text{Machine Learning} \supset \text{Artificial Neural Networks} \supset \text{Deep Learning}}$$

으로 생각할 수 있다.

여기서 한 가지 주의할 점은 artificial neural network와 deep learning이 완전히 같은 의미는 아니라는 것이다. 하나 또는 소수의 layer만을 갖는 neural network도 존재하며, deep learning은 일반적으로 여러 layer를 갖는 neural network를 이용하여 복잡한 함수를 학습하는 방법을 의미한다.

Artificial Neural Network는 어디에서 시작되었는가?

오늘날 인공지능 연구의 중요한 발전을 이끌고 있는 deep learning의 뿌리는 오래전부터 연구되어 온 artificial neural network(ANN)에 있다. Artificial neural network는 생물학적인 신경망에서 영감을 받아 만들어진 computational model이다.

"스스로 학습하고 판단하는 기계"를 만드는 것은 오랫동안 인간의 꿈이었다. 초기의 인공지능 연구자들은 인간의 뇌가 정보를 처리하는 방식에서 아이디어를 얻어, 뇌의 기본 단위인 신경세포, 즉 neuron에 관심을 가졌다.

생물학적인 neuron은 다른 neuron으로부터 여러 신호를 받아들이고, 이 신호들을 종합한 뒤 일정한 조건이 만족되면 다시 다른 neuron으로 신호를 전달한다. 중요한 점은 인간의 뇌가 하나의 neuron만으로 복잡한 판단을 수행하는 것이 아니라, 매우 많은 neuron이 서로 연결된 network를 이용한다는 것이다.

이러한 아이디어로부터 다음과 같은 단순한 computational unit을 생각할 수 있다.

$$a = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b,$$

그리고 이 weighted sum에 어떤 함수 ϕ를 적용하여

$$z = \phi(a)$$

를 출력하도록 한다.

즉,

$$\boxed{z = \phi(w_1 x_1 + \cdots + w_n x_n + b)}$$

와 같은 매우 단순한 계산을 하나의 artificial neuron이 수행한다고 생각한다.

여기서 $x_1, \ldots, x_n$은 다른 neuron 또는 외부로부터 들어오는 입력신호이고, $w_1, \ldots, w_n$은 각 입

력신호의 중요도를 나타내는 weight, b는 bias이다. 함수 ϕ는 입력신호의 weighted sum을 실제 출력신호로 변환하며 activation function이라고 부른다.

Artificial neural network는 이러한 단순한 unit 또는 node를 많이 연결하여 만든다. 각 node의 출력은 다시 다른 node의 입력이 되고, 이러한 계산이 여러 layer를 따라 반복된다.

$$\text{input} \longrightarrow \text{simple computations} \longrightarrow \text{simple computations} \longrightarrow \cdots \longrightarrow \text{output}.$$

하나의 neuron이 할 수 있는 계산은 매우 단순하다. 그러나 많은 neuron을 서로 연결하고 그 사이에 nonlinear activation을 넣으면 전체 network는 매우 복잡한 함수를 표현할 수 있다.

바로 이 점이 neural network의 핵심 아이디어이다.

Neural network를 하나의 거대한 함수로 보기

Neural network를 생물학적인 뇌의 모형으로만 생각할 필요는 없다. 수학적으로는 많은 간단한 함수들을 반복적으로 합성하여 만들어진 하나의 매우 복잡한 함수라고 생각할 수 있다.

$$\boxed{\text{Neural Network} = \text{Composition of many simple functions}}$$

그리고 learning이란 이 함수 안에 들어 있는 수많은 weight와 bias를 데이터로부터 결정하는 과정이다.

1 Artificial Neuron과 Perceptron

1.1 하나의 neuron은 무엇을 계산하는가?

입력 vector를

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

라고 하자.

하나의 artificial neuron은 먼저 입력의 가중합(weighted sum)

$$\boxed{a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b = \sum_{j=1}^{n} w_j x_j + b}$$

을 계산하고, 그 결과에 activation function ϕ를 적용하여

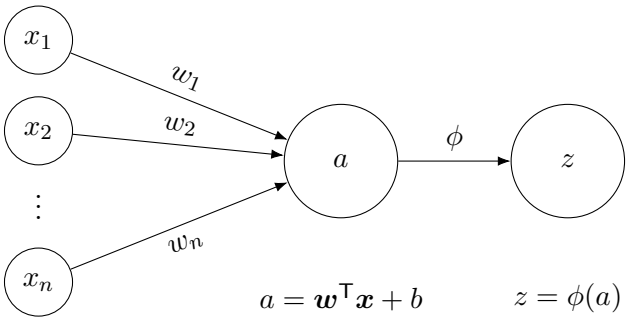
$$\boxed{z = \phi(a)}$$

를 출력한다.

따라서 하나의 neuron은

$$\boxed{\boldsymbol{x} \longrightarrow a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b \longrightarrow z = \phi(a)}$$

라는 두 단계의 합성함수로 볼 수 있다.



여기서 w_j는 각 입력 x_j가 neuron에 미치는 영향의 크기를 정하고, b는 전체 weighted sum을 이동시키는 역할을 한다.

1.2 Classical perceptron

고전적인 perceptron에서는 activation function으로 step function을 사용한다.

$$H(a) = \begin{cases} 0, & a \le 0, \\ 1, & a > 0. \end{cases}$$

따라서 perceptron의 출력은

$$\boxed{z = H\big(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b\big)}$$

이다.

즉,

$$\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b > 0$$

이면 1을 출력하고 그렇지 않으면 0을 출력한다.

Perceptron과 modern artificial neuron

고전적인 의미의 perceptron은 step activation을 사용하는 threshold classifier를 가리킨다. 현대의 neural network에서는 같은 weighted-sum 구조를 유지하면서 step function 대신 sigmoid, tanh, ReLU와 같은 activation function을 사용한다. 이 강의에서는 필요에 따라 "neuron"이라는 더 일반적인 표현을 사용한다.

1.3 1차원에서는 무엇을 하는가?

입력이 하나라면

$$a = wx + b$$

이고 perceptron은

$$z = H(wx + b)$$

이다.

$w \neq 0$ 일 때 출력이 바뀌는 위치는

$$wx + b = 0$$

이므로

$$\boxed{x = -\frac{b}{w}}$$

이다.

즉 1차원 perceptron은 실수축을 하나의 경계점으로 나누어 두 class로 분류한다.

2 Activation Function

2.1 왜 step function만으로는 학습하기 어려운가?

step function은 $a = 0$에서 미분가능하지 않고, $a \neq 0$에서는

$$H'(a) = 0$$

이다.

gradient descent로 parameter를 학습하려면

$$\frac{\partial L}{\partial w}, \qquad \frac{\partial L}{\partial b}$$

와 같은 derivative가 필요하다. 그러나 activation의 derivative가 거의 모든 곳에서 0이면 gradient가 앞쪽 parameter로 전달되지 않는다.

따라서 gradient-based learning을 위해서는 **미분가능하거나 거의 모든 곳에서 유용한 derivative를 갖는 activation function**이 필요하다.

2.2 Sigmoid function

가장 대표적인 smooth activation 중 하나는 logistic sigmoid이다.

$$\boxed{\sigma(a) = \frac{1}{1 + e^{-a}}}$$

sigmoid는 모든 실수 입력을 $(0, 1)$ 사이의 값으로 보낸다.

특히 derivative는

$$\boxed{\sigma'(a) = \sigma(a)\big(1 - \sigma(a)\big)}$$

이다.

따라서 forward에서

$$z = \sigma(a)$$

를 이미 계산해 두었다면 backward에서는

$$\boxed{\frac{dz}{da} = z(1-z)}$$

로 바로 사용할 수 있다.

Forward에서 계산한 값을 backward에서 다시 사용한다

앞 주차에서 보았듯 backpropagation은 graph를 거꾸로 따라가며 local derivative를 곱한다. sigmoid에서는 forward에서 계산한 z 자체가

$$\frac{dz}{da} = z(1-z)$$

를 계산하는 데 사용된다. 실제 neural network framework에서도 forward 과정의 중간값을 저장해 두었다가 backward에서 재사용하는 구조가 중요하다.

2.3 Tanh와 ReLU

자주 사용되는 activation을 몇 가지 정리하면 다음과 같다.

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}, \qquad \boxed{\frac{d}{da}\tanh(a) = 1 - \tanh^2(a)}$$

그리고 ReLU(Rectified Linear Unit)는

$$\boxed{\mathrm{ReLU}(a) = \max(0, a)}$$

이고 $a \neq 0$에서

$$\mathrm{ReLU}'(a) = \begin{cases} 0, & a < 0, \\ 1, & a > 0. \end{cases}$$

Activation	Output range	Derivative	특징
Step	$\{0, 1\}$	0 a.e.	classical perceptron
Sigmoid	$(0, 1)$	$z(1-z)$	binary output에 직관적
Tanh	$(-1, 1)$	$1 - z^2$	zero-centered output
ReLU	$[0, \infty)$	0 or 1	modern hidden layer에 널리 사용

이 주차에서는 sigmoid를 중심으로 계산하고, tanh와 ReLU는 이후 network 구조를 다룰 때 다시 사용한다.

3 왜 Nonlinearity가 필요한가?

activation function의 가장 중요한 수학적 역할은 단순히 생물학적 neuron의 "활성/비활성"을 흉내 내는 것에 있지 않다. 핵심은 network에 **nonlinearity**를 넣는 것이다.

두 affine transformation을 생각하자.

$$\boldsymbol{u} = W_1\boldsymbol{x} + \boldsymbol{b}_1, \qquad \boldsymbol{y} = W_2\boldsymbol{u} + \boldsymbol{b}_2.$$

이를 합성하면

$$\begin{aligned}\boldsymbol{y} &= W_2(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \\ &= (W_2W_1)\boldsymbol{x} + (W_2\boldsymbol{b}_1 + \boldsymbol{b}_2).\end{aligned}$$

즉,

$$\boxed{\text{affine} \circ \text{affine} = \text{affine}}$$

이다.

affine layer를 2개, 20개, 200개 쌓더라도 중간에 nonlinear activation이 없다면 전체 mapping은 결국 하나의 affine mapping과 같다.

Activation function의 수학적 역할

Neural network가 복잡한 함수를 표현하려면

$$\boxed{\text{Affine transformation} \quad + \quad \text{Nonlinear activation}}$$

의 반복이 필요하다.
Network의 "깊이"만 늘리는 것으로는 충분하지 않다. 비선형성이 들어가야 표현력이 증가한다.

4 하나의 Sigmoid Neuron으로 Binary Classification

4.1 Binary classification

다음 데이터를 생각하자.

x_i	-2	-1	0	1	2
t_i	0	0	1	1	1

target t_i가 0 또는 1의 두 값만 가지므로 binary classification 문제이다.

하나의 sigmoid neuron을 사용하여

$$\boxed{a_i = wx_i + b, \qquad z_i = \sigma(a_i)}$$

를 예측모델로 사용하자.

z_i는 $(0, 1)$ 사이의 연속적인 값을 출력한다. 이를 class label로 바꾸기 위해 예를 들어

$$\widehat{t}_i = \begin{cases} 1, & z_i \geq 0.5, \\ 0, & z_i < 0.5 \end{cases}$$

로 정할 수 있다.

sigmoid는

$$\sigma(0) = 0.5$$

이고 증가함수이므로

$$z_i \geq 0.5 \quad \Longleftrightarrow \quad wx_i + b \geq 0.$$

따라서 decision boundary는

$$\boxed{x = -\frac{b}{w}} \qquad (w \neq 0)$$

이다.

4.2 Computational graph

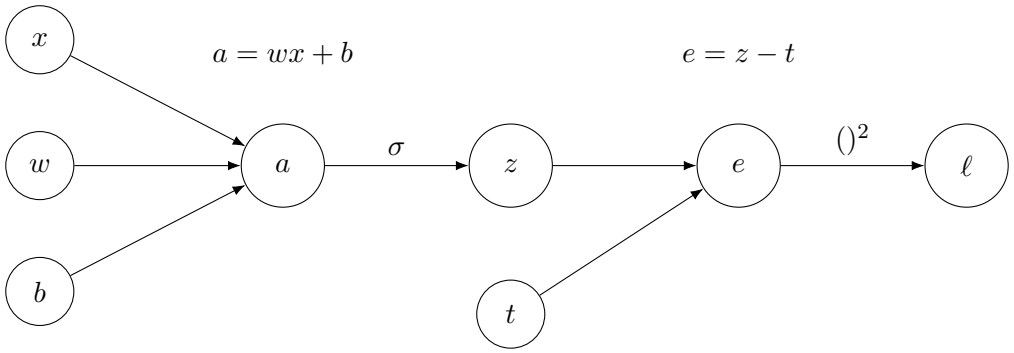
한 개의 datum (x, t)에 대해 다음 변수를 정의하자.

$$a = wx + b, \qquad z = \sigma(a), \qquad e = z - t, \qquad \ell = e^2.$$

그러면 computational graph는

$$(x, w, b) \longrightarrow a \longrightarrow z \longrightarrow e \longrightarrow \ell$$

이다.



4.3 Backward: local derivative를 따라가기

각 edge에서 필요한 local derivative는

$$\frac{\partial \ell}{\partial e} = 2e, \qquad \frac{\partial e}{\partial z} = 1, \qquad \frac{\partial z}{\partial a} = z(1 - z),$$
$$\frac{\partial a}{\partial w} = x, \qquad \frac{\partial a}{\partial b} = 1.$$

따라서 chain rule에 의해

$$\boxed{\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial e}\frac{\partial e}{\partial z}\frac{\partial z}{\partial a}\frac{\partial a}{\partial w} = 2e\, z(1-z)x}$$

이고

$$\boxed{\frac{\partial \ell}{\partial b} = 2e\, z(1-z)}$$

이다.

즉 linear output 뒤에 sigmoid node 하나가 추가되면서 backward에서도 local derivative

$$z(1-z)$$

하나가 추가된 것이다.

Backpropagation의 관점

Forward:

$$x, w, b \longrightarrow a \longrightarrow z \longrightarrow e \longrightarrow \ell.$$

Backward:

$$\frac{\partial \ell}{\partial e} \longrightarrow \frac{\partial \ell}{\partial z} \longrightarrow \frac{\partial \ell}{\partial a} \longrightarrow \left(\frac{\partial \ell}{\partial w}, \frac{\partial \ell}{\partial b}\right).$$

Network가 길어져도 원리는 같다. 뒤쪽에서 전달된 gradient에 현재 node의 local derivative를 곱하여 앞쪽으로 전달한다.

4.4 여러 데이터에 대한 gradient

N 개의 데이터 (x_i, t_i)에 대해 이 주차에서는 계산 구조를 명확하게 보기 위해 MSE loss를 사용하자.

$$L(w,b) = \frac{1}{N}\sum_{i=1}^{N}(z_i - t_i)^2, \qquad z_i = \sigma(wx_i + b).$$

그러면

$$\boxed{\frac{\partial L}{\partial w} = \frac{2}{N}\sum_{i=1}^{N}(z_i - t_i)z_i(1-z_i)x_i}$$

$$\boxed{\frac{\partial L}{\partial b} = \frac{2}{N}\sum_{i=1}^{N}(z_i - t_i)z_i(1-z_i)}$$

이고 gradient descent는

$$\begin{bmatrix} w \\ b \end{bmatrix} \leftarrow \begin{bmatrix} w \\ b \end{bmatrix} - \eta \begin{bmatrix} \partial L/\partial w \\ \partial L/\partial b \end{bmatrix}$$

로 수행한다.

Classification loss에 대한 참고

Binary classification에서는 binary cross-entropy(BCE)가 더 표준적인 loss이다. 하지만 이번 주에는 activation node가 추가되었을 때 chain rule과 backward가 어떻게 바뀌는지를 선명하게 보기 위해 MSE를 사용한다. BCE와 softmax/cross-entropy는 뒤에서 별도로 다룬다.

5 Single Neuron의 한계

하나의 sigmoid neuron은 1차원에서

$$z(x) = \sigma(wx + b)$$

형태이다.

$w > 0$이면 증가함수이고, $w < 0$이면 감소함수이다. 따라서 하나의 sigmoid neuron은 본질적으로 하나의 transition만 만들 수 있다.

예를 들어 target이

x	-2	-1	0	1	2
t	0	1	1	1	0

처럼 "가운데만 class 1"이라면 하나의 threshold로는 분류할 수 없다. class $0 \to 1$로 바뀌는 경계와 $1 \to 0$으로 바뀌는 경계가 두 개 필요하기 때문이다.

마찬가지로 하나의 sigmoid

$$\sigma(wx + b)$$

만으로는 x^2, $\sin x$와 같은 일반적인 non-monotone function을 근사하기 어렵다.

무엇이 더 필요한가?

하나의 neuron이 만드는 단순한 비선형 함수를 여러 개 만들어서 이들을 다시 조합하면 어떨까?
이 생각이 바로 **hidden layer**의 출발점이다.

6 Hidden Layer와 Nonlinear Function Approximation

6.1 하나의 hidden layer

입력 $x \in \mathbb{R}$에서 m개의 hidden neurons를 만드는 network를 생각하자.

각 hidden neuron에서

$$a_j = x\, w_j^{(1)} + b_j^{(1)}, \qquad h_j = \phi(a_j), \qquad j = 1, \ldots, m$$

를 계산하고, 마지막 출력에서는 이 hidden outputs를 다시 선형결합하여

$$\boxed{\widehat{y} = \sum_{j=1}^{m} h_j \, w_j^{(2)} + b^{(2)}}$$

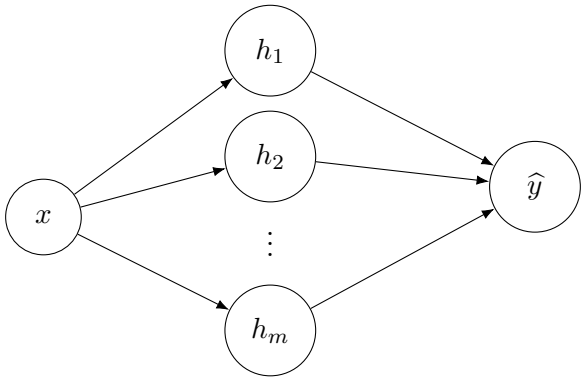
를 계산한다.

즉,

$$\boxed{\widehat{y}(x) = \sum_{j=1}^{m} \phi\Big(x \, w_j^{(1)} + b_j^{(1)}\Big) \, w_j^{(2)} + b^{(2)}}$$

이다.

$$h_j = \phi(x \, w_j^{(1)} + b_j^{(1)})$$



입력과 출력 사이에 직접 관측되지 않는 중간 variables

$$h_1, \ldots, h_m$$

가 존재하므로 이 층을 **hidden layer**라고 부른다.

6.2 Matrix form

입력 x를 앞에, weight matrix를 뒤에 곱하는 **행벡터(row-vector) 규약**을 사용하자. (NumPy, PyTorch 코드의 `X @ W + b`와 같은 순서이다.)

$$W^{(1)} = \begin{bmatrix} w_1^{(1)} & \cdots & w_m^{(1)} \end{bmatrix} \in \mathbb{R}^{1\times m}, \qquad \boldsymbol{b}^{(1)} = \begin{bmatrix} b_1^{(1)} & \cdots & b_m^{(1)} \end{bmatrix} \in \mathbb{R}^{1\times m}$$

라고 두면

$$\boldsymbol{a} = x \, W^{(1)} + \boldsymbol{b}^{(1)} \in \mathbb{R}^{1\times m}, \qquad \boldsymbol{h} = \phi(\boldsymbol{a}) \in \mathbb{R}^{1\times m}$$

이고 (ϕ는 성분별로 적용),

$$\widehat{y} = \boldsymbol{h}\,\boldsymbol{w}^{(2)} + b^{(2)}, \qquad \boldsymbol{w}^{(2)} = \begin{bmatrix} w_1^{(2)} \\ \vdots \\ w_m^{(2)} \end{bmatrix} \in \mathbb{R}^{m\times 1}$$

로 compact하게 쓸 수 있다.

따라서 전체 network는

$$\boxed{x \longrightarrow \boldsymbol{a} \longrightarrow \boldsymbol{h} \longrightarrow \widehat{y}}$$

라는 합성함수이다.

6.3 Hidden neuron은 서로 다른 nonlinear basis를 만든다

sigmoid activation을 사용하면 각 hidden neuron은

$$h_j(x) = \sigma\Big(w_j^{(1)}x + b_j^{(1)}\Big)$$

와 같은 S-shaped function을 만든다.

$w_j^{(1)}$ 와 $b_j^{(1)}$ 가 달라지면 각 sigmoid의 기울기와 transition 위치가 달라진다.

마지막 output layer는 이 함수들을

$$w_1^{(2)}h_1(x) + \cdots + w_m^{(2)}h_m(x) + b^{(2)}$$

와 같이 선형결합한다.

즉 hidden layer의 neuron들은 서로 다른 nonlinear "building blocks"를 만들고, output layer가 이들을 조합하여 더 복잡한 함수를 만든다.

7 Universal Approximation의 직관

7.1 Sigmoid 두 개로 구간을 만들기

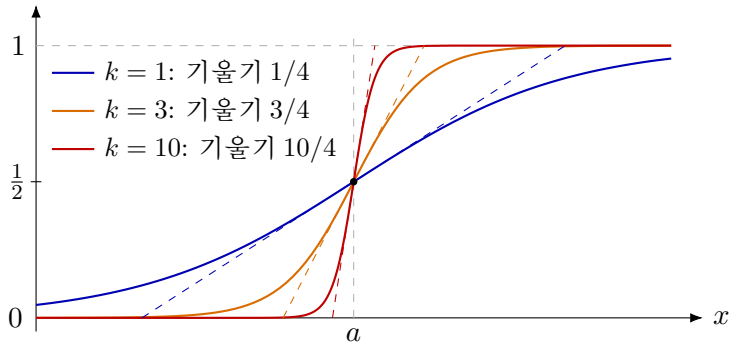
먼저 sigmoid 하나를 평행이동하고 가로 방향으로 압축한 함수

$$\sigma(k(x-a)), \qquad k>0$$

를 살펴보자. hidden neuron $\sigma(x\,w+b)$에서 $w>0$이면

$$x\,w + b = w\left(x - \left(-\tfrac{b}{w}\right)\right)$$

이므로, $k=w$, $a=-b/w$ 인 경우에 해당한다.



실선: $\sigma(k(x-a))$, 점선: $x=a$에서의 접선

모든 k에 대하여 $\sigma(k(x-a))$는 점 $\left(a, \frac{1}{2}\right)$를 지난다. 즉 a는 함수가 0 쪽에서 1 쪽으로 넘어가는 **위치**를 정하고, k는 그 변화가 얼마나 **가파른지**를 정한다.

이를 미분으로 확인하자. $z = k(x-a)$라 두면 $\frac{dz}{dx} = k$이고, sigmoid의 derivative $\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$와 chain rule에 의해

$$\frac{d}{dx}\sigma(k(x-a)) = \sigma'(z)\,\frac{dz}{dx} = k\,\sigma(k(x-a))\big(1-\sigma(k(x-a))\big)$$

이다. $x=a$에서는 $z=0$이고 $\sigma(0)=\frac{1}{2}$이므로

$$\boxed{\left.\frac{d}{dx}\sigma(k(x-a))\right|_{x=a} = k\cdot\frac{1}{2}\cdot\left(1-\frac{1}{2}\right) = \frac{k}{4}}$$

를 얻는다.

따라서 $x=a$에서의 접선은

$$y = \frac{1}{2} + \frac{k}{4}(x-a)$$

이고, 이 접선은 $x = a - \frac{2}{k}$에서 $y=0$, $x = a + \frac{2}{k}$에서 $y=1$이 된다. 즉 함수가 0 근처에서 1 근처로 바뀌는 구간의 폭은 대략 $\frac{4}{k}$이다.

k가 커지면 sigmoid는 step function에 가까워진다

$x=a$에서의 기울기 $\frac{k}{4}$는 k에 비례하여 커지고, 전이 구간의 폭 $\frac{4}{k}$는 0으로 줄어든다. 실제로 $k \to \infty$이면

$$\sigma(k(x-a)) \longrightarrow \begin{cases} 0, & x < a, \\ \frac{1}{2}, & x = a, \\ 1, & x > a \end{cases}$$

이므로, 큰 k에 대한 sigmoid는 $x=a$에서 뛰어오르는 step function처럼 행동한다.

정리하면, 큰 양수 k에 대하여

$$\sigma(k(x-a))$$

는 $x=a$ 근처에서 0에서 1로 빠르게 변한다.

따라서 두 sigmoid의 차

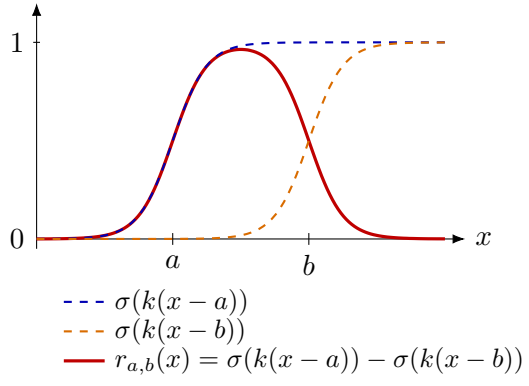
$$\boxed{r_{a,b}(x) = \sigma(k(x-a)) - \sigma(k(x-b))} \qquad (a < b)$$

는 k가 충분히 클 때 대략

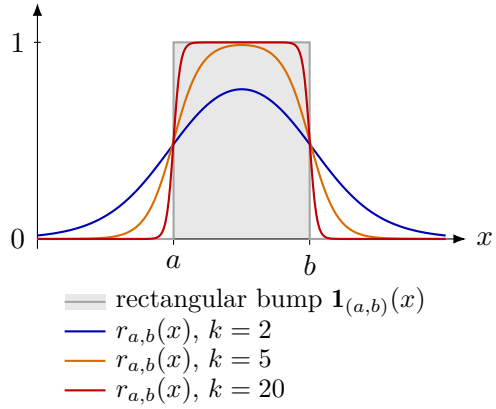
$$r_{a,b}(x) \approx \begin{cases} 1, & a < x < b, \\ 0, & \text{otherwise} \end{cases}$$

와 같은 rectangular bump를 만든다.

(i) 두 sigmoid의 차 ($k = 4$)



(ii) k가 커질수록 rectangular bump에 가까워진다



즉 hidden neuron들의 선형결합만으로도 특정 구간에서만 값이 큰 함수를 만들 수 있다.

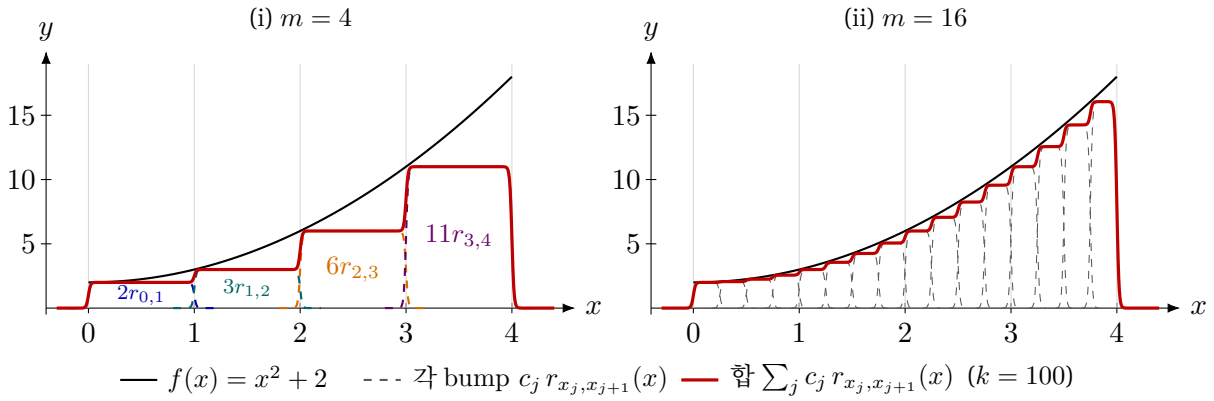
7.2 Rectangular bumps를 더하면 일반적인 함수를 근사할 수 있다

연속함수 f가 구간 $[a, b]$에 있다고 하자. 구간을 작은 조각들로 나누고 각 구간에서 f의 값을 하나의 상수로 근사하면 piecewise constant function을 얻는다.

각 piece는 앞의 rectangular bump로 근사할 수 있으므로

$$f(x) \approx \sum_{j=1}^{m} c_j \left[\sigma(k(x-a_j)) - \sigma(k(x-a_{j+1}))\right].$$

예를 들어 $f(x) = x^2 + 2$를 $[0, 4]$에서 같은 폭의 m개 구간 $[x_j, x_{j+1}]$ $(x_j = 4j/m)$으로 나누고, 각 구간의 왼쪽 끝 값 $c_j = f(x_j)$를 높이로 사용하자. $k = 100$으로 고정하고 각 bump $c_j\, r_{x_j, x_{j+1}}(x)$를 더하면(superposition) 아래와 같이 f를 계단 모양으로 근사한다. 왼쪽($m = 4$, $c = 2, 3, 6, 11$)은 3차시 실습 Step 7과 같은 구간 설정이다.



$k = 100$이면 각 bump는 거의 완전한 직사각형이므로, 남아 있는 f와의 차이는 k가 아니라 **구간의 폭**에서 온다. 왼쪽 끝 값을 사용하면 구간 $[x_j, x_{j+1}]$에서의 오차는 대략 $f(x_{j+1}) - f(x_j) \approx f'(x_j) \cdot \frac{4}{m}$이므로, $m = 4$일 때 최대 약 7이던 오차가 $m = 16$에서는 약 2로 줄어든다. 즉 bump의 개수, 곧 **hidden neuron의 개수를 늘려야** f에 더 가까워진다.

우변을 정리하면 결국

$$\boxed{f(x) \approx \sum_{j=1}^{M} \alpha_j \sigma(\beta_j x + \gamma_j) + c}$$

와 같은 형태가 된다.

그런데 이것이 바로 one-hidden-layer neural network의 출력형태이다.

Universal Approximation Theorem — 1차원 형태

구간 $[a, b]$에서 연속인 함수 f와 임의의 $\varepsilon > 0$가 주어졌다고 하자. 적절한 조건을 만족하는 nonlinear activation(예: sigmoid)을 사용하면, 충분히 많은 hidden neurons를 갖는 one-hidden-layer network

$$N(x) = \sum_{j=1}^{m} \alpha_j \sigma(\beta_j x + \gamma_j) + c$$

를 선택하여

$$\boxed{\sup_{x \in [a,b]} |f(x) - N(x)| < \varepsilon}$$

가 되도록 할 수 있다.

즉 hidden layer 하나만으로도 충분히 많은 neuron이 있다면 매우 다양한 continuous function을 임의의 정확도로 근사할 수 있다.

7.3 중요한 주의: 존재와 학습은 다른 문제이다

Universal Approximation Theorem이 말하는 것은

$$\boxed{\text{그러한 parameter가 존재한다}}$$

는 것이다.

그러나 이것만으로

$$\boxed{\text{gradient descent가 그 parameter를 쉽게 찾아낸다}}$$

는 것은 보장되지 않는다.

Representability ≠ Trainability

Universal approximation은 network의 **표현력(expressive power)**에 대한 정리이다.
실제 learning에서는 별개의 문제가 남는다.

- 어떤 loss를 사용할 것인가?
- parameter를 어떻게 초기화할 것인가?
- gradient를 어떻게 효율적으로 계산할 것인가?
- gradient descent가 좋은 parameter를 찾을 수 있는가?

다음 단계에서는 hidden layer를 가진 network에서 backpropagation으로 이 많은 parameter의 gradient를 어떻게 계산하는지를 다룬다.

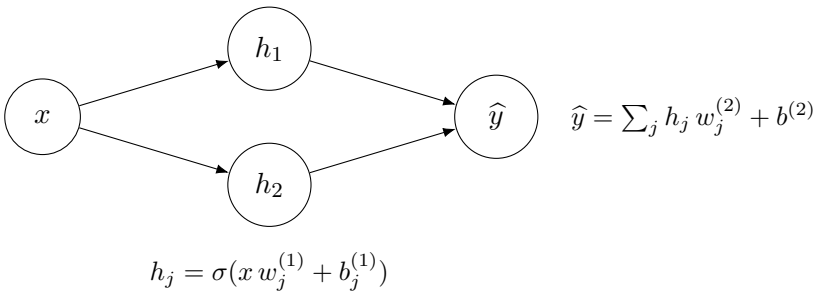
8 3차시 실습: NumPy Hard Coding

이번 실습에서도 PyTorch의 autograd를 사용하지 않는다. 이번 주의 목표는 hidden layer를 하나 가진 network에 대해

$$\boxed{\text{input} \to \text{hidden layer} \to \text{output} \to \text{loss} \to \text{backward} \to \text{update}}$$

의 구조를 코드에서 직접 확인하는 것이다.

사용할 network는 6절의 one-hidden-layer network와 같다: 입력 x 하나, sigmoid hidden neuron $m = 2$개, 선형 출력 $\hat{y}$ 하나.



데이터는 x축 위에 순서대로 놓인 네 개의 class $t \in \{0, 1, 2, 3\}$이다. class 번호를 target 값으로 두고 MSE로 학습한 뒤, 출력 $\hat{y}$를 가장 가까운 정수로 반올림하여 class를 예측한다. target이 계단 모양이므로, 7절에서 본 것처럼 여러 sigmoid를 조합해야 표현할 수 있다.

8.1 Step 1. 데이터와 parameter

N 개의 데이터를 한 줄에 하나씩 쌓아 **데이터 행렬**로 만든다. `reshape(-1, 1)`은 길이 N 인 1차원 배열을 $N \times 1$ 행렬(열벡터)로 바꾼다.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

# data: N rows = N samples, 4 classes ordered along x
x = np.array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6],
             dtype=float).reshape(-1, 1)       # X: (N, 1)
t = np.array([ 0,  0,  0,  0,  1, 1, 1, 2, 2, 3, 3, 3],
             dtype=float).reshape(-1, 1)       # T: (N, 1)
N = x.shape[0]

m = 2   # number of hidden neurons

# parameters (row-vector convention: a = x W1 + b1,  y = h W2 + b2)
W1 = np.random.randn(1, m) * 0.5 + 0.1   # (1, m)
b1 = np.random.randn(1, m) * 0.5 + 0.1   # (1, m)
W2 = np.random.randn(m, 1) * 0.5         # (m, 1)
b2 = np.random.randn(1, 1) * 0.5         # (1, 1)

learning_rate = 0.2
epochs = 20000

def sigmoid(a):
    return 1.0 / (1.0 + np.exp(-a))
```

8.2 Step 2. Forward

전체 데이터를 한 번에 network에 흘려보낸다.

```
a = x @ W1 + b1      # (N,1) @ (1,m) + (1,m) -> (N, m)
h = sigmoid(a)       # (N, m)
y = h @ W2 + b2      # (N,m) @ (m,1) + (1,1) -> (N, 1)

error = y - t        # (N, 1)
loss = np.mean(error**2)
```

데이터 i 하나에 대해서는 6절과 똑같이

$$a_{ij} = x_i\, w_j^{(1)} + b_j^{(1)}, \qquad h_{ij} = \sigma(a_{ij}), \qquad y_i = \sum_{j=1}^{m} h_{ij}\, w_j^{(2)} + b^{(2)}, \qquad e_i = y_i - t_i$$

를 계산하고, loss는 $L = \frac{1}{N}\sum_{i=1}^{N} e_i^2$ 이다. 데이터 행렬

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^{N\times 1}, \qquad T = \begin{bmatrix} t_1 \\ \vdots \\ t_N \end{bmatrix} \in \mathbb{R}^{N\times 1}$$

로 한꺼번에 쓰면

$$A = XW^{(1)} + \mathbf{1}\,\boldsymbol{b}^{(1)}, \qquad H = \sigma(A), \qquad \boldsymbol{y} = H\boldsymbol{w}^{(2)} + \mathbf{1}\,b^{(2)}, \qquad \boldsymbol{e} = \boldsymbol{y} - T$$

이다. $A, H \in \mathbb{R}^{N\times m}$의 i번째 행이 6.2절의 $\boldsymbol{a}$, $\boldsymbol{h}$, 즉 데이터 i의 hidden layer이다. ($\mathbf{1} \in \mathbb{R}^{N\times 1}$은 모든 성분이 1인 벡터이며, NumPy에서는 `+ b1`, `+ b2`의 broadcasting이 이 역할을 한다.)

8.3 Step 3. Backward — local derivative를 그대로 코드로 쓰기

```
# Row i of every array below belongs to sample i.
# So each row is exactly the single-sample backprop,
# computed for all N samples at the same time.

# ---- output layer:  y_i = sum_j h_ij * w2_j + b2 ----
dL_dy = 2.0 * error / N       # (N, 1)  L = mean(e**2) -> 2 e_i / N

# W2, b2 are SHARED by all N samples -> sum of contributions
dL_dW2 = h.T @ dL_dy                              # (m,N)@(N,1) -> (m, 1)
dL_db2 = np.sum(dL_dy, axis=0, keepdims=True)  # (1, 1)

# ---- back to the hidden layer:  dy_i/dh_ij = w2_j ----
dL_dh = dL_dy @ W2.T          # (N,1)@(1,m) -> (N, m)

# ---- through the sigmoid (elementwise):  dh/da = h (1 - h) ----
dL_da = dL_dh * h * (1.0 - h) # (N, m)

# W1, b1 are SHARED by all N samples -> sum of contributions
dL_dW1 = x.T @ dL_da                              # (1,N)@(N,m) -> (1, m)
dL_db1 = np.sum(dL_da, axis=0, keepdims=True)  # (1, m)

# Because dL_dy already contains 1/N, these sums are the
# AVERAGE of the per-sample gradients.
```

데이터 i 하나에 대한 backward는 출력에서 입력 방향으로 local derivative를 차례로 곱하는 것이다:

$$\delta_i := \frac{\partial L}{\partial y_i} = \frac{2e_i}{N}, \qquad \frac{\partial L}{\partial h_{ij}} = \delta_i \underbrace{\frac{\partial y_i}{\partial h_{ij}}}_{w_j^{(2)}}, \qquad \delta_{ij} := \frac{\partial L}{\partial a_{ij}} = \frac{\partial L}{\partial h_{ij}} \underbrace{\frac{\partial h_{ij}}{\partial a_{ij}}}_{h_{ij}(1-h_{ij})}.$$

기호 δ와 Δ: 각 node에 도착한 gradient

backpropagation에서 δ는 관례적으로 "loss L을 어떤 node의 값으로 미분한 것", 즉 **출력 쪽에서 그 node까지 거꾸로 전달되어 온 gradient**를 나타낸다. 이 값만 알면 그 node로 들어오는 parameter의 gradient는 local derivative 하나를 더 곱해서 바로 얻을 수 있다.

- $\delta_i = \dfrac{\partial L}{\partial y_i}$: 데이터 i의 **출력 node** y_i에 도착한 gradient (스칼라).
- $\delta_{ij} = \dfrac{\partial L}{\partial a_{ij}}$: 데이터 i의 j**번째 hidden neuron의 입력** a_{ij} (activation을 통과하기 전 값)에 도착한 gradient (스칼라).

이들을 N개의 데이터에 대해 모아 행렬로 쓴 것이

$$\boldsymbol{\delta} = \begin{bmatrix} \delta_1 \\ \vdots \\ \delta_N \end{bmatrix} \in \mathbb{R}^{N\times 1}, \qquad \Delta = \begin{bmatrix} \delta_{11} & \cdots & \delta_{1m} \\ \vdots & \ddots & \vdots \\ \delta_{N1} & \cdots & \delta_{Nm} \end{bmatrix} \in \mathbb{R}^{N\times m}$$

이다. (Δ는 대문자 delta이며, 여기서는 "변화량"이 아니라 δ_{ij}를 모은 행렬을 뜻한다.) Δ의 i번째 행은 데이터 i의 hidden layer 전체에 도착한 gradient이다.

기호	의미	코드
$\boldsymbol{\delta} \in \mathbb{R}^{N\times 1}$	$\partial L/\partial \boldsymbol{y}$ (출력 node)	`dL_dy`
$\Delta \in \mathbb{R}^{N\times m}$	$\partial L/\partial A$ (hidden neuron의 입력)	`dL_da`

parameter에 대해서는

$$\frac{\partial L}{\partial w_j^{(2)}} = \sum_{i=1}^{N} h_{ij}\,\delta_i, \qquad \frac{\partial L}{\partial b^{(2)}} = \sum_{i=1}^{N} \delta_i, \qquad \frac{\partial L}{\partial w_j^{(1)}} = \sum_{i=1}^{N} x_i\,\delta_{ij}, \qquad \frac{\partial L}{\partial b_j^{(1)}} = \sum_{i=1}^{N} \delta_{ij}$$

이다. 이를 행렬로 쓰면 ($\odot$는 성분별 곱)

$$\text{output layer:}\quad \boldsymbol{\delta} = \tfrac{2}{N}\boldsymbol{e}, \qquad \frac{\partial L}{\partial \boldsymbol{w}^{(2)}} = H^\top \boldsymbol{\delta}, \qquad \frac{\partial L}{\partial b^{(2)}} = \mathbf{1}^\top \boldsymbol{\delta}$$

$$\text{hidden layer:}\quad \Delta = \left(\boldsymbol{\delta}\,(\boldsymbol{w}^{(2)})^\top\right) \odot H \odot (1-H), \qquad \frac{\partial L}{\partial W^{(1)}} = X^\top \Delta, \qquad \frac{\partial L}{\partial \boldsymbol{b}^{(1)}} = \mathbf{1}^\top \Delta$$

이며, 코드의 `h.T @ dL_dy`, `dL_dy @ W2.T`, `x.T @ dL_da`, `np.sum(..., axis=0)`가 정확히 이것이다.

왜 bias의 gradient는 합(sum)인가?

앞 절의 backpropagation은 데이터 **하나**에 대한 계산이었다. 코드에서는 N 개의 데이터를 한 번에 흘려보내므로, `dL_dy`, `dL_da`의 i 번째 행이 곧 데이터 i 하나에 대한 δ_i, δ_{ij} 이다.
그런데 parameter는 데이터마다 따로 있는 것이 아니라 **모든 데이터가 공유**한다. 예를 들어 출력층의 bias는

$$y_1 = \textstyle\sum_j h_{1j} w_j^{(2)} + b^{(2)}, \quad \ldots, \quad y_N = \sum_j h_{Nj} w_j^{(2)} + b^{(2)}$$

처럼 $y_1, \ldots, y_N$ 모두에 들어간다 (forward에서 $b^{(2)}$ 가 N 개의 행으로 broadcast된다). 따라서 chain rule에 의해 각 경로의 기여를 모두 더해야 한다:

$$\frac{\partial L}{\partial b^{(2)}} = \sum_{i=1}^{N} \frac{\partial L}{\partial y_i} \underbrace{\frac{\partial y_i}{\partial b^{(2)}}}_{1} = \sum_{i=1}^{N} \delta_i.$$

$\boldsymbol{b}^{(1)}$, $W^{(1)}$, $\boldsymbol{w}^{(2)}$ 도 같은 이유로 데이터에 대한 합이 되고, weight의 경우 그 합을 행렬곱 $X^\top \Delta$, $H^\top \boldsymbol{\delta}$ 로 한 번에 계산한다. $\boldsymbol{\delta}$ 안에 $1/N$ 이 들어 있으므로 이 합은 곧 **데이터별 gradient의 평균**이며, parameter는 epoch마다 한 번 업데이트된다 (full-batch gradient descent).

8.4 Step 4. Parameter update

```
W1 -= learning_rate * dL_dW1
b1 -= learning_rate * dL_db1
W2 -= learning_rate * dL_dW2
b2 -= learning_rate * dL_db2
```

즉, 모든 parameter $\theta \in \{W^{(1)}, \boldsymbol{b}^{(1)}, \boldsymbol{w}^{(2)}, b^{(2)}\}$ 에 대해

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

8.5 Step 5. 전체 training loop

```
import os
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
os.makedirs("figures", exist_ok=True)   # folder for saved figures

x = np.array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6],
             dtype=float).reshape(-1, 1)       # (N, 1)
t = np.array([ 0,  0,  0,  0,  1, 1, 1, 2, 2, 3, 3, 3],
             dtype=float).reshape(-1, 1)       # (N, 1)
N = x.shape[0]

m = 2
W1 = np.random.randn(1, m) * 0.5 + 0.1
```

```
b1 = np.random.randn(1, m) * 0.5 + 0.1
W2 = np.random.randn(m, 1) * 0.5
b2 = np.random.randn(1, 1) * 0.5

learning_rate = 0.2
epochs = 20000
loss_history = []

def sigmoid(a):
    return 1.0 / (1.0 + np.exp(-a))

for epoch in range(epochs):

    # ------------------------------
    # 1. Forward (all N samples at once)
    # ------------------------------
    a = x @ W1 + b1            # (N, m)
    h = sigmoid(a)             # (N, m)
    y = h @ W2 + b2            # (N, 1)

    error = y - t
    loss = np.mean(error**2)

    # ------------------------------
    # 2. Backward (row i = single-sample backprop of sample i)
    # ------------------------------
    dL_dy = 2.0 * error / N                          # (N, 1)
    dL_dW2 = h.T @ dL_dy                             # (m, 1)  sum over samples
    dL_db2 = np.sum(dL_dy, axis=0, keepdims=True)    # (1, 1)  sum over samples

    dL_dh = dL_dy @ W2.T                             # (N, m)
    dL_da = dL_dh * h * (1.0 - h)                    # (N, m)
    dL_dW1 = x.T @ dL_da                             # (1, m)  sum over samples
    dL_db1 = np.sum(dL_da, axis=0, keepdims=True)    # (1, m)  sum over samples

    # ------------------------------
    # 3. Gradient descent (one update per epoch)
    # ------------------------------
    W1 -= learning_rate * dL_dW1
    b1 -= learning_rate * dL_db1
    W2 -= learning_rate * dL_dW2
    b2 -= learning_rate * dL_db2

    loss_history.append(loss)

    if epoch % 2000 == 0:
        print(f"epoch={epoch:5d}, loss={loss:.6f}")

print("W1 =", W1)
print("b1 =", b1)
print("W2 =", W2.ravel())
```

```
print("b2 =", b2)

# prediction curve
x_plot = np.linspace(-7.0, 8.0, 300).reshape(-1, 1)   # (300, 1)
h_plot = sigmoid(x_plot @ W1 + b1)                    # (300, m)
y_plot = h_plot @ W2 + b2                             # (300, 1)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(x, t, "ro", label="data")
plt.plot(x_plot, y_plot, "k", label="network output")
plt.xlabel("x")
plt.ylabel("y")
plt.title("2-Layer Neural Network")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(loss_history)
plt.xlabel("epoch")
plt.ylabel("MSE loss")
plt.title("Cost History")
plt.grid(True)
plt.tight_layout()
plt.savefig("figures/lab_fit_and_cost.png", dpi=200)
plt.show()

# what does each hidden neuron do?
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
for j in range(m):
    plt.plot(x_plot, h_plot[:, j], "--", label=f"h_{j+1}")
plt.title("hidden layer outputs")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
for j in range(m):
    plt.plot(x_plot, h_plot[:, j] * W2[j, 0], "--", label=f"h_{j+1} * w2_{j+1}")
plt.plot(x_plot, y_plot, "k", label="y = sum + b2")
plt.title("output layer")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("figures/lab_hidden_units.png", dpi=200)
plt.show()
```

학습 후에는 network 출력 $\hat{y}(x)$가 데이터의 계단 모양을 따라가는 곡선이 된다. 그림은 `figures` 폴더에 png 파일로도 저장된다.

[그림 자리] `figures/lab_fit_and_cost.png`
Step 5 코드를 실행하면 생성된다.

아래 그림에서 각 hidden neuron의 출력 $h_j(x)$는 서로 다른 위치 $-b_j^{(1)}/w_j^{(1)}$와 기울기를 가진 S-shaped curve이고, 출력층은 이들을 $w_j^{(2)}$배 하여 더한 뒤 $b^{(2)}$만큼 평행이동한다.

[그림 자리] `figures/lab_hidden_units.png`
Step 5 코드를 실행하면 생성된다.

관찰해 보자

데이터의 계단은 세 개이지만 hidden neuron은 두 개뿐이다. 학습된 두 neuron은 대체로 다음과 같이 역할을 나눈다.

- 한 neuron은 $x \approx -1.5$ 근처에서 **가파르게** 전이하여 높이 1의 첫 번째 계단을 만든다.
- 다른 neuron은 $x \approx 2.5$ 근처를 중심으로 **완만하게** 전이하여, 높이 약 2의 S-shaped curve 하나로 위쪽 두 계단($1 \to 2 \to 3$)을 함께 따라간다.

데이터가 정수 위치에만 있으므로, 완만한 sigmoid 하나로도 두 계단을 충분히 지나갈 수 있는 것이다. 즉 7절처럼 계단마다 sigmoid를 하나씩 배정하지 않아도 된다. `m`을 3, 5로 늘리면 training loss는 조금 줄지만 곡선에 불필요한 굴곡이 생기는지 관찰하라. (과제 2와 7.3절: 표현 가능성과 학습 가능성은 다른 문제이다.)

8.6 Step 6. 분류 결과 확인

```
a = x @ W1 + b1
h = sigmoid(a)
y = h @ W2 + b2

# nearest class index (0, 1, 2, 3)
prediction = np.clip(np.rint(y), 0, 3).astype(int)

print("network output =", np.round(y.ravel(), 3))
print("prediction     =", prediction.ravel())
print("target         =", t.ravel().astype(int))
```

8.7 Step 7. Hidden neurons가 만드는 basis function 관찰

이번에는 training을 하지 않고, 여러 sigmoid를 조합하면 복잡한 함수를 만들 수 있다는 사실만 확인해보자.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(a):
    return 1.0 / (1.0 + np.exp(-a))
```

```

x = np.linspace(-0.5, 4.5, 500)

k = 20.0

# four approximate rectangular bumps
r0 = sigmoid(k*(x-0)) - sigmoid(k*(x-1))
r1 = sigmoid(k*(x-1)) - sigmoid(k*(x-2))
r2 = sigmoid(k*(x-2)) - sigmoid(k*(x-3))
r3 = sigmoid(k*(x-3)) - sigmoid(k*(x-4))

# left-endpoint values of f(x)=x^2+2
f_approx = 2*r0 + 3*r1 + 6*r2 + 11*r3
f_true = x**2 + 2

plt.figure(figsize=(7, 4))
plt.plot(x, f_true, label="f(x)=x^2+2")
plt.plot(x, f_approx, label="sigmoid combination")
plt.xlim(0, 4)
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

여기서 각 r_j는 sigmoid 두 개의 차로 만든 approximate rectangular bump이고, 이들을 서로 다른 계수로 더하여 $x^2 + 2$를 거칠게 근사하였다.

hidden neurons의 수를 늘리고 interval을 더 잘게 나누면 더 정밀한 approximation을 만들 수 있다.

9 이번 주의 핵심 정리

핵심 구조

하나의 neuron:

$$\boxed{a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b, \qquad z = \phi(a)}$$

Activation의 역할:

$$\boxed{\text{network에 nonlinearity를 넣는다}}$$

Single sigmoid neuron:

$$\boxed{z = \sigma(wx + b)}$$

는 하나의 transition을 표현한다.

Hidden layer:

$$\boxed{\widehat{y}(x) = \sum_{j=1}^{m} w_j^{(2)} \phi(w_j^{(1)} x + b_j^{(1)}) + b^{(2)}}$$

는 여러 nonlinear building blocks를 조합한다.

Backpropagation:

$$\boxed{\text{뒤쪽 gradient} \times \text{local derivative} \longrightarrow \text{앞쪽 gradient}}$$

Universal approximation:

$$\boxed{\text{충분히 많은 hidden neurons} \Rightarrow \text{매우 풍부한 함수 표현력}}$$

10 연습문제

1. $$a = 2x - 1, \qquad z = \sigma(a)$$

   에서 $x = 1$일 때 $a, z, dz/da, dz/dx$를 계산하라.

2. 하나의 neuron

   $$a = w_1 x_1 + w_2 x_2 + b, \qquad z = \sigma(a)$$

   에 대해

   $$\nabla_{\boldsymbol{w}} z$$

   를 구하라.

3. sigmoid neuron에서 $w > 0$일 때

   $$z(x) = \sigma(wx + b) = 0.5$$

   가 되는 x를 구하고, w와 b가 decision boundary에 어떤 영향을 주는지 설명하라.

4. $$\ell = (z - t)^2, \qquad z = \sigma(wx + b)$$

에 대해 computational graph를 그리고

$$\frac{\partial \ell}{\partial w}, \qquad \frac{\partial \ell}{\partial b}$$

를 local derivative의 곱으로 유도하라.

5. 활성화 함수가 identity,

$$\phi(a) = a,$$

라고 하자. 두 affine layer

$$\boldsymbol{h} = W_1 \boldsymbol{x} + \boldsymbol{b}_1, \qquad \boldsymbol{y} = W_2 \boldsymbol{h} + \boldsymbol{b}_2$$

를 하나의 affine layer로 합칠 수 있음을 보이라.

6. 하나의 sigmoid neuron으로

$$(0, 1, 1, 1, 0)$$

과 같이 가운데에서만 class 1인 1차원 classification pattern을 완벽하게 표현하기 어려운 이유를 설명하라.

7.

$$r_{a,b}(x) = \sigma(k(x-a)) - \sigma(k(x-b))$$

에서 k가 매우 커질 때 함수의 모양이 어떻게 변하는지 설명하라.

8. Universal Approximation Theorem이 "gradient descent가 항상 좋은 solution을 찾는다"는 것을 의미하지 않는 이유를 설명하라.

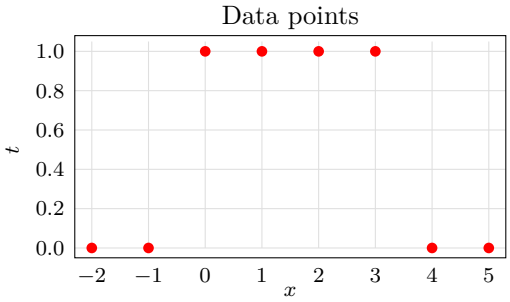
11 과제

과제 1은 hidden layer가 있는 network의 순전파와 역전파를 **손으로** 끝까지 계산해 보는 문제이고, 과제 2는 3차시 실습 코드로 network를 **설계하고 훈련**해 보는 문제이다.

과제 1. 은닉층이 있는 신경망의 역전파를 손으로 계산하기

다음과 같은 데이터가 주어졌다고 하자.

i	x	t
0	-2	0
1	-1	0
2	0	1
3	1	1
4	2	1
5	3	1
6	4	0
7	5	0



은닉층이 1층이고 두 개의 노드를 갖는 함수로 주어진 데이터를 회귀하고자 한다.

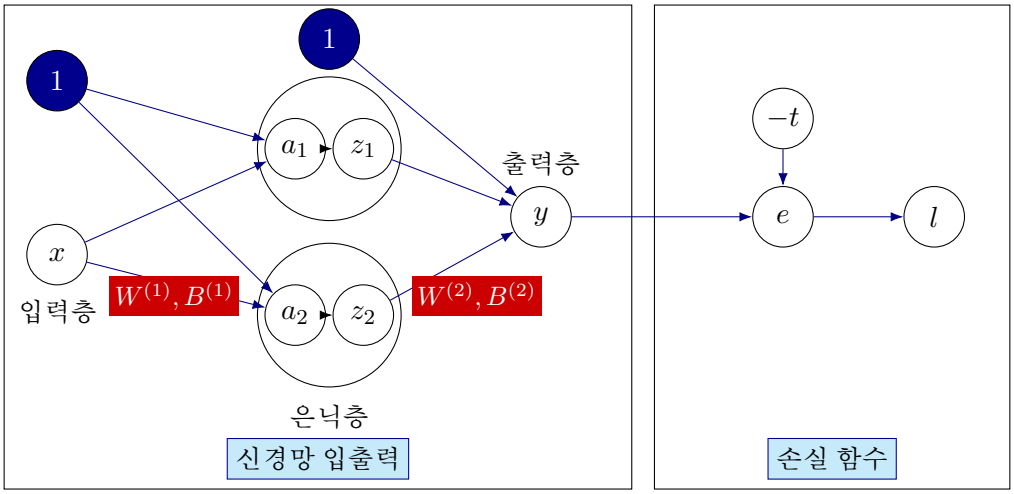


그림 4

활성화함수 $h(x) = \max(0, x)$ (ReLU)일 때, 각 노드에 나타나는 변수들을

$$\begin{cases} a_1 = xw_1^{(1)} + b_1^{(1)} \\ a_2 = xw_2^{(1)} + b_2^{(1)} \end{cases}, \quad \begin{cases} z_1 = h(a_1) \\ z_2 = h(a_2) \end{cases}, \quad y = z_1 w_1^{(2)} + z_2 w_2^{(2)} + b^{(2)}$$

로 정의하면

$$y = w_1^{(2)} h\left(w_1^{(1)} x + b_1^{(1)}\right) + w_2^{(2)} h\left(w_2^{(1)} x + b_2^{(1)}\right) + b^{(2)}$$

이고, 가중치와 편이와 은닉층의 변수를 행렬로

$$W_{1\times 2}^{(1)} = \left(w_1^{(1)}, w_2^{(1)}\right), \quad B_{1\times 2}^{(1)} = \left(b_1^{(1)}, b_2^{(1)}\right), \quad W_{2\times 1}^{(2)} = \begin{pmatrix} w_1^{(2)} \\ w_2^{(2)} \end{pmatrix}, \quad B_{1\times 1}^{(2)} = \left(b^{(2)}\right)$$

$$A = (a_1, a_2), \quad Z = (z_1, z_2)$$

로 정의하면, 신경망은 다음과 같은 단계를 따라 출력 y를 계산한다.

$$A = (x)_{1\times 1} W^{(1)} + B^{(1)}, \quad Z = h(A), \quad y = ZW^{(2)} + B^{(2)}$$

신경망의 출력 y와 실제 데이터 t의 차이인 오차를 $e = y - t$이라 정의하고, 제곱 오차를 $l = e^2$이라 하자. 모든 데이터에 대한 평균제곱 손실함수(MSE)는

$$L\left(W^{(1)}, B^{(1)}, W^{(2)}, B^{(2)}\right) = \frac{1}{N} \sum_{i=0}^{N-1} l_i$$

이다.

ReLU의 도함수

ReLU의 도함수는

$$h'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

이다. 즉 입력이 양수인 노드는 gradient를 그대로 통과시키고, 입력이 음수인 노드는 gradient를 0으로 막는다. (꺾이는 점 $x = 0$에서는 미분이 정의되지 않으므로 보통 $h'(0) = 0$으로 약속한다. NumPy 코드의 `np.where(a > 0, 1, 0)`과 같다.)

데이터 $(x_3, t_3) = (1, 1)$와 가중치와 편이의 초깃값

$$W_0^{(1)} = (1, 0), \quad B_0^{(1)} = (0, 1), \quad W_0^{(2)} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad B_0^{(2)} = 0$$

에 대하여

가. 순전파로 정해지는 변수값 A, Z, y, e, l의 값을 구하여라.

나. 역전파로 정해지는 편미분

$$\frac{\partial l}{\partial e}, \frac{\partial l}{\partial y}, \frac{\partial l}{\partial W_0^{(2)}}, \frac{\partial l}{\partial B_0^{(2)}}, \frac{\partial l}{\partial Z}, \frac{\partial l}{\partial A}, \frac{\partial l}{\partial W_0^{(1)}}, \frac{\partial l}{\partial B_0^{(1)}}, \frac{\partial l}{\partial x}$$

의 값을 구하여라.

다. $\alpha = 0.01$일 때, 경사하강법을 적용하여

$$\begin{cases} W_1^{(1)} = W_0^{(1)} - \alpha \dfrac{\partial l}{\partial W_0^{(1)}} \\ B_1^{(1)} = B_0^{(1)} - \alpha \dfrac{\partial l}{\partial B_0^{(1)}} \\ W_1^{(2)} = W_0^{(2)} - \alpha \dfrac{\partial l}{\partial W_0^{(2)}} \\ B_1^{(2)} = B_0^{(2)} - \alpha \dfrac{\partial l}{\partial B_0^{(2)}} \end{cases}$$

$W_1^{(1)}, B_1^{(1)}, W_1^{(2)}, B_1^{(2)}$를 구하여라.

라. 만약 초깃값이 $B_0^{(1)} = (0, -1)$이었다면 (나머지 초깃값과 데이터는 같다), 나.의 편미분 중 어떤 값이 0이 되는가? 이때 두 번째 은닉 노드의 가중치 $w_2^{(1)}$, 편이 $b_2^{(1)}$, 가중치 $w_2^{(2)}$는 경사하강법으로 업데이트되는가? 만약 **모든** 데이터에 대해 $a_2 < 0$이라면 학습이 진행되어도 어떤 일이 일어나겠는가?

과제 2. 신경망 설계와 훈련 실험

3차시 실습 Step 5의 코드(`hidden_layer_lab.py`)를 활용하여,

```
x = np.array([-2,-1,0,1,2,3,4,5,-5,-4,-3,-1.5])
t = np.array([ 0, 0,1,1,1,1,0,0,1,1,1,0])
```

를 회귀하는 신경망을 훈련시켜 훈련된 가중치와 편이를 출력하고, 3차시 실습 Step 5와 같은 그래프(학습 결과와 Cost History, hidden layer의 출력)를 출력하여라. 이 때, 은닉층의 노드 수, 학습률, epochs 등, 훈련 파라미터를 조정하여 여러 실험을 통해 가능한 최적의 결과를 도출하여라.

다음 순서로 실험하고, 각 단계의 그래프와 함께 관찰한 내용을 서술하라.

가. **은닉층의 노드 수.** 먼저 노드 수 `m = 2`로 훈련해 보라. 데이터를 잘 회귀하는가? x가 증가할 때 t가 0과 1 사이를 몇 번 바꾸는지 세어 보고, sigmoid hidden neuron 하나가 만들 수 있는 전이(transition)의 개수(7절)와 비교하여 이 데이터에 **필요한 최소 노드 수**를 추론하라. 그 다음 `m`을 늘려 추론이 맞는지 확인하라.

나. **학습률.** 가.에서 정한 `m`을 고정하고 `learning_rate`를 0.01, 0.05, 0.2, 0.5, 1.0으로 바꾸어 Cost History를 비교하라. 학습률이 너무 작으면, 또 너무 크면 각각 어떤 일이 일어나는가? (loss가 `nan`이 되는 경우도 관찰하라.) 같은 epochs 안에 안정적으로 수렴하는 학습률의 범위는 어느 정도인가?

다. **노드가 너무 많으면?** `m = 20`, `epochs = 100000`으로 훈련해 보라.

- 최종 training loss를 가.의 결과와 비교하라.
- 학습된 곡선을 `x_plot` 구간 전체에서 그려, 데이터 점 **사이**와 데이터 **바깥**에서 곡선이 어떤 모양인지 가.의 곡선과 비교하라.
- 훈련에 쓰지 않은 새로운 x에 대해서는 두 network 중 어느 쪽의 예측을 더 믿을 수 있는가?

training loss는 작지만 새로운 입력에 대한 예측이 나빠지는 현상을 **과적합(overfitting)**이라 한다. 왜 노드 수가 많을수록 이런 일이 일어나기 쉬운지 parameter의 개수와 데이터의 개수를 비교하여 설명하라.

라. **최종 선택.** 위 실험을 바탕으로 선택한 (노드 수, 학습률, epochs)와 그 근거를 쓰고, 훈련된 $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$와 최종 그래프를 제출하라.